JalSathi

Rainwater Harvesting and Water Conservation Monitor: A Bilingual Decision-Support Platform for Small and Marginal Farmers of Maharashtra

Submitted in partial fulfillment of the requirements

for the degree of

Bachelor of Technology

in Information Technology

by

Roll No.	Name
02	Ayush Jage
62	Heramb Chopde
____	Melwin _____
57	Vinayak Sahu

Branch / Division: SEIT **Batch: C**

Supervisor:

Asst. Prof. Shiv Negi



UNIVERSITY OF MUMBAI

2026-2027

JalSathi

Rainwater Harvesting and Water Conservation Monitor: A Bilingual Decision-Support Platform for Small and Marginal Farmers of Maharashtra

Submitted in partial fulfillment of the requirements

of the degree of

Bachelor of Technology

by

Roll No.	Name
02	Ayush Jage
62	Heramb Chopde
____	Melwin _____
57	Vinayak Sahu

Branch / Division: SEIT Batch: C

Supervisor:
Asst. Prof. Shiv Negi

Department of Information Technology

Don Bosco Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

2026-2027

DON BOSCO INSTITUTE OF TECHNOLOGY

Vidyavihar Station Road, Mumbai - 400070

Department of Information Technology

CERTIFICATE

This is to certify that the project entitled **"JalSathi"** is a bonafide work of

Ayush Jage	02
Heramb Chopde	62
Melwin _____	_____
Vinayak Sahu	57

submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** in **Information Technology** (Branch: SEIT, Batch: C).

Date: / /

(Asst. Prof. Shiv Negi)
Supervisor

(Prof. Sunantha G.)
HOD, IT Department

(Dr. Sudhakar Mande)
Principal

DON BOSCO INSTITUTE OF TECHNOLOGY

Vidyavihar Station Road, Mumbai - 400070

Department of Information Technology

Project Report Approval

This project report entitled **"JalSathi"** by **Ayush Jage, Heramb Chopde, Melwin _____ and Vinayak Sahu** is approved for the degree of **Bachelor of Technology in Information Technology**

(Examiner's Name and Signature)

1. _____

2. _____

(Supervisor's Name and Signature)

1. _____

(Chairman)

1. _____

Date:

Place:

DON BOSCO INSTITUTE OF TECHNOLOGY
Vidyavihar Station Road, Mumbai - 400070

Department of Information Technology

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Ayush Jage (Roll No. 02)

Heramb Chopde (Roll No. 62)

Melwin _____ (Roll No. ____)

Vinayak Sahu (Roll No. 57)

Date:

Abstract

Groundwater depletion and erratic monsoons hit small and marginal farmers across Maharashtra's 36 districts hardest, yet a field survey found that 67% of them have no rainwater harvesting (RWH) structure. The causes are unpredictable rainfall, missing technical knowledge, a high perceived cost without a clear payback, and neglected maintenance. Existing help is mostly generic advisories and static guides that give a farmer no way to size a structure, test the numbers or see a payback.

This project, *JalSathi* (Paani Mitra), is a bilingual (Marathi default, English toggle), mobile-first web application that works as a decision-support and monitoring platform. A farmer can compute harvestable water from catchment area, rainfall and runoff coefficient, track a tank with an inflow and outflow ledger, and run a 365-day tank simulation on real daily rainfall. The platform also scores soil fertility from 0 to 100, grades water against BIS IS:10500, recommends crops and RWH structures, estimates tanker savings and payback, and shows India WRIS groundwater wells on a map. A Weka machine-learning module trained on 115 years of IMD rainfall forecasts the season. A big-button assisted mode lets a field worker (Krushi Sahayak) register farmers with low digital literacy, and an admin console with PDF and CSV export supports monitoring.

The backend is a Spring Boot 3.2.5 REST service (Java 17) with Spring Data JPA, stateless JWT security and BCrypt hashing, backed by MySQL 8 or a file-based H2 database with 23 tables and 8 bundled datasets. The frontend uses plain HTML5, CSS3 and ES6 JavaScript with Chart.js and Leaflet.js. The project maps to UN SDG 6, 13 and 15 and shows how a smartphone can replace costly field equipment for water-conservation planning.

Keywords: Rainwater Harvesting, Water Conservation, Decision Support System, Machine Learning, Weka, Spring Boot, REST API, GIS, Precision Agriculture, SDG 6

Contents

1 Introduction **1**
- 1.1 Problem Statement . . . 1
- 1.2 Scope of the Project . . . 2
- 1.3 Current Scenario . . . 2
- 1.4 Need for the Proposed System . . . 3
- 1.5 Summary of the Results and Tasks Completed . . . 3

2 Review of Literature **4**
- 2.1 Theoretical Background . . . 4
  - 2.1.1 Rainwater Harvesting and Runoff . . . 4
  - 2.1.2 RWH Structures Used by Small Farmers . . . 4
  - 2.1.3 Water Balance of a Storage Tank . . . 5
  - 2.1.4 Soil Fertility and Water Quality . . . 5
  - 2.1.5 Machine Learning with Weka . . . 5
  - 2.1.6 Geographic Information Systems . . . 5
  - 2.1.7 Web Application Architecture . . . 6
  - 2.1.8 Relational Databases and MySQL . . . 6
  - 2.1.9 Internationalisation and Inclusive Design . . . 6
- 2.2 Summary of the Investigation in the Published Sources . . . 6
- 2.3 Comparison between the Tools, Methods and Algorithms . . . 7
- 2.4 Algorithm(s) with Example . . . 8
  - 2.4.1 Harvest Calculation . . . 8
  - 2.4.2 365-Day Tank Simulation . . . 8
  - 2.4.3 Soil Fertility Index . . . 8
  - 2.4.4 Water-Quality Verdict . . . 9
  - 2.4.5 Payback Period . . . 9

3 Analysis and Design **10**
- 3.1 Methodology / Procedure Adopted . . . 10
- 3.2 Analysis . . . 10
  - 3.2.1 Feasibility Study . . . 10
  - 3.2.2 Modification of Requirements . . . 11
  - 3.2.3 Software / System Requirement Specification – IEEE Format . . . 11
- 3.3 System Architecture / Design . . . 11
  - 3.3.1 Modules and their Description . . . 12

3.4 Detailed Design Rationale . . . 16
3.4.1 Frontend Design . . . 16
3.4.2 Authentication and Role-Based Access . . . 17
3.4.3 Backend Layered Design . . . 17
3.4.4 Sequence of a Harvest Calculation . . . 17
3.4.5 Sequence of a Monsoon Forecast . . . 17
3.4.6 Non-Functional Design Decisions . . . 18

4 Implementation **19**
4.1 Implementation Plan . . . 19
4.2 Coding Standard . . . 20
4.2.1 Naming Conventions . . . 20
4.2.2 Comments, Structure and Complexity . . . 20
4.3 Testing . . . 22
4.3.1 Testing Methods Used . . . 22
4.3.2 Test Cases for Modules and Components . . . 22
4.3.3 Results of Testing and System Performance . . . 23

5 Results and Discussion **25**
5.1 Intermediate Results . . . 25
5.2 Final Results and Analysis . . . 25
5.2.1 Landing Page, Login and Language Toggle . . . 25
5.2.2 Farmer Dashboard . . . 25
5.2.3 Runoff and Harvesting Calculator . . . 25
5.2.4 Rainfall Logger and District Normals . . . 27
5.2.5 Tank Storage Tracker . . . 28
5.2.6 365-Day Tank Simulation . . . 28
5.2.7 Soil Fertility Index . . . 29
5.2.8 Structure Feasibility and Recommendation . . . 29
5.2.9 AI Climate and Crop Water-Budget Advisor . . . 30
5.2.10 GIS Groundwater Monitoring . . . 31
5.2.11 Water Quality Assessment . . . 31
5.2.12 Economic Payback and Subsidies . . . 32
5.2.13 Knowledge Base, Community Movements and Research Evidence . . . 32
5.2.14 Assisted Mode for Field Workers . . . 33
5.2.15 Administrator Console . . . 33
5.2.16 Reports and Alerts . . . 34
5.2.17 Comparison: Existing Practice and Proposed System . . . 34
5.2.18 Interpretation of the Results . . . 36
5.2.19 Discussion and Limitations . . . 36

6 Conclusion and Future Work **37**
6.1 Conclusion . . . 37
6.2 Learning from Project Management . . . 37
6.3 Future Work . . . 38

A Software Requirement Specification (IEEE Format) 39
A.1 Introduction . . . 39
A.1.1 Purpose . . . 39
A.1.2 Scope . . . 39
A.1.3 Definitions . . . 39
A.2 Overall Description . . . 40
A.2.1 Product Perspective . . . 40
A.2.2 User Classes and Characteristics . . . 40
A.2.3 Operating Environment . . . 40
A.2.4 Assumptions and Dependencies . . . 40
A.3 Specific Requirements . . . 40
A.3.1 Functional Requirements . . . 40
A.3.2 Non-Functional Requirements . . . 41

B Installation Procedure: Development Software 42
B.1 Software Required . . . 42
B.2 Running the Application . . . 42
B.3 Demonstration Accounts . . . 43

C Data Sheets: Electronic Components 44

References 45

Acknowledgement 48

List of Figures

3.1 Three-tier architecture of JalSathi 12
3.2 Use case diagram of JalSathi 14
3.3 Entity-relationship diagram of JalSathi 14

4.1 Gantt chart of the JalSathi project (12 weeks) 19
4.2 REST response captured in Swagger UI 24

5.1 Landing and login page (Marathi default) 26
5.2 Language toggle showing the English interface 26
5.3 Farmer dashboard 27
5.4 Runoff and harvesting calculator 27
5.5 Daily rainfall log against district normals 28
5.6 Tank tracker with gauge and transaction ledger 28
5.7 365-day simulation and capacity sweep 29
5.8 Soil fertility index and advice 29
5.9 Structure feasibility and dual-track recommendation 30
5.10 AI climate and crop water-budget advisor 30
5.11 Groundwater observation wells on the Leaflet map 31
5.12 Water-quality test and BIS IS:10500 verdict 31
5.13 Cost, payback and subsidy calculator 32
5.14 Illustrated guides in the knowledge base 32
5.15 Community movements and research evidence 33
5.16 Big-button assisted registration wizard 33
5.17 Administrator dashboard 34
5.18 Exported PDF water-budget report 34
5.19 Smart alerts and notifications 35

List of Tables

1.1 Module groups implemented in JalSathi 3

2.1 Comparison of candidate technologies 7

3.1 Functional modules of JalSathi 12
3.2 Main database entities 15
3.3 REST controllers exposed by the backend 16

4.1 Work Breakdown Structure and individual contribution 19
4.2 Naming conventions used in the project 21
4.3 Test cases for modules and components 22

5.1 Comparison of existing practice and JalSathi 35

B.1 Seeded demonstration accounts 43

Chapter 1

Introduction

1.1 Problem Statement

Maharashtra's agriculture depends on the monsoon. Dry spells of 20 to 30 days, recurring droughts in Marathwada and Vidarbha, and heavy groundwater extraction push small and marginal farmers toward expensive private water tankers. A baseline survey of such farmers found that 67% of them have no rainwater harvesting (RWH) structure at all, even though seasonal rain could meet much of their irrigation and domestic need. Four causes stand out:

1. **Rainfall unpredictability:** intense rain events are followed by long dry spells, and over half of the post-monsoon runoff drains away unharvested [19].
2. **Lack of local technical knowledge:** catchments and structures are sized by guesswork, so pits are too small or wash out.
3. **High perceived cost:** farmers assume RWH costs lakhs of rupees, although a lined Jalkund can cost under Rs. 8,000 with a payback of under two years [16].
4. **Maintenance neglect:** siltation can cut farm-pond capacity by 10 to 15% a year without pre-monsoon desilting.

The aim of this project is to design and develop JalSathi, a smartphone-friendly web platform in which a farmer or field worker can:

- calculate harvestable rainwater for any catchment using a standard hydrological formula;
- track a storage tank with a double-entry inflow and outflow ledger;
- simulate a full year of tank behaviour and test alternative tank sizes;
- measure soil fertility and water quality and receive advice in Marathi or English;
- get crop, structure and seasonal water-budget recommendations backed by machine learning;
- estimate tanker savings, payback period and eligible government schemes; and
- allow an administrator to monitor adoption and export reports.

1.2 Scope of the Project

The scope of the project is limited to a functional prototype that covers the full planning-to-monitoring workflow for a farmer. The following assumptions were made during requirement gathering and problem definition:

- The target users are small and marginal farmers (below 2 to 5 acres) and rural field workers, many with low digital literacy.
- The system needs no sensors or special equipment; the farmer enters rainfall and tank readings.
- The interface is bilingual: Marathi by default with an instant English toggle.
- Reference data (rainfall, groundwater, soil, crops, water-quality norms) is bundled with the application and loaded at start-up; no live external API is called yet.
- IoT sensors, live weather feeds, an IVR or WhatsApp bot and MahaDBT integration are left as future work.

The work was carried out in the following phases:

- Requirements Gathering Phase: studying the farmer survey, the barriers to RWH adoption and the features needed to remove them.
- Requirements Analysis Phase: writing the software requirement specification, defining functional and non-functional requirements and listing the use cases.
- System Design Phase: designing the three-tier architecture, the 23-table database, the REST API and the screen flow.
- Implementation and Testing Phase: building the backend, the datasets, the machine-learning module and the web screens, and verifying them with automated and manual tests.

1.3 Current Scenario

Farmers today rely on three main sources of help. Government schemes such as *Magel Tyala Shettale* and *Jalyukt Shivar Abhiyan* offer funding, but the farmer still needs technical planning to apply. Community movements such as the Paani Foundation's Water Cup and the NAAM Foundation's desilting drives mobilise whole villages through shramdaan [11, 12]. Local Krushi Kendras and weather advisories give generic guidance.

These existing practices have clear limitations. Advisories are generic and do not use the farmer's own plot size or soil. Community drives work at village scale and cannot tell one farmer what size of pit or pond suits his land. Lab reports on soil and water are not interpreted for the farmer. None of these lets a single farmer, on a phone, size a structure for his own plot, see how it behaves over a year, and learn what it would save.

1.4 Need for the Proposed System

A farmer adopts RWH only when he can see the water, the cost and the payback for his own land. JalSathi turns abstract advice into numbers: litres harvested, days of shortage avoided, rupees saved, and a plain-language Safe, Caution or Not Safe verdict on his water. A Marathi interface and an assisted mode remove the language and literacy barrier. For administrators and NGOs, the same data becomes taluka-level adoption statistics. The platform directly supports UN SDG 6 (clean water and sanitation), SDG 13 (climate action) and SDG 15 (life on land) [14].

1.5 Summary of the Results and Tasks Completed

The project delivers 20 functional modules on a Spring Boot REST backend with 20 controllers, 23 database tables and 8 bundled datasets, together with a vanilla web frontend. The 14 automated backend tests pass with no failures or errors. Table 1.1 lists the main module groups.

Table 1.1: Module groups implemented in JalSathi

No.	Module group	Main features
1	Identity and access	JWT login, three roles (Farmer, Field Worker, Admin), 36-district registration
2	Water core	Runoff calculator, tank ledger, 365-day simulation
3	Agronomy and quality	Soil Fertility Index, BIS IS:10500 water-quality verdict, structure recommender
4	AI and analytics	Weka monsoon forecast, crop recommendation, seasonal water budget
5	GIS and knowledge	India WRIS groundwater map, guides, community movements, research evidence
6	Economics and alerts	Tanker and pumping savings, payback, subsidies, smart alerts
7	Inclusion and administration	Assisted registration wizard, admin dashboard, PDF and CSV reports, SDG mapping

The implemented system therefore meets the aim stated in Section 1.1 at the level of a working prototype: planning, simulation, advice, economics, monitoring and administration are all available end to end, while live weather, sensors and messaging are reserved for future work.

Chapter 2

Review of Literature

2.1 Theoretical Background

This section explains the concepts on which the project is based. It is written so that a reader who is new to hydrology, agronomy or web development can follow the rest of the report.

2.1.1 Rainwater Harvesting and Runoff

Rainwater harvesting is the collection and storage of rainfall from a catchment (a roof, a yard or a field) for later use. The volume that can be collected depends on three things: the catchment area, the rainfall depth, and the runoff coefficient C, which says what fraction of the rain actually runs off rather than soaking into the ground or evaporating. The coefficients used in the platform are:

- **Concrete or rooftop:** 0.85
- **Compacted earthen yard:** 0.60
- **Cultivated farmland:** 0.40
- **Pasture or grassland:** 0.30
- **Forested or vegetated land:** 0.20

The harvested volume is then

$$\text{Harvested Litres} = \text{Area}\,(\text{m}^2) \times \text{Rainfall}\,(\text{mm}) \times C.$$

Because 1 mm of rain on 1 m^2 equals 1 litre, the product is directly in litres. Farmers in Maharashtra measure land in gunthas and acres, so the system converts 1 guntha = 101.17 m^2 and 1 acre = 4046.86 m^2 before applying the formula.

2.1.2 RWH Structures Used by Small Farmers

The platform recommends from the following structures, which are the ones most used in rural Maharashtra:

- **Jalkund:** a small pit lined with plastic sheet that costs roughly Rs. 3,000 to Rs. 8,000 and suits a low budget.

- **Farm pond:** a larger excavated pond, often lined with a 500-micron geotextile or plastic liner.
- **Recharge pit and shaft:** a sand-and-gravel filter pit around a borewell or well that sends runoff into the aquifer.
- **Masonry or HDPE tank:** closed storage for rooftop water, usually with a first-flush separator that discards the dirty first rain.
- **Check dam and contour trench:** community-scale structures used in watershed programmes.

2.1.3 Water Balance of a Storage Tank

A tank can be modelled day by day. The stored volume on day t equals the previous volume plus the inflow from rain, minus the demand, limited between zero and the tank capacity:

$$S_t = \min\left(C_{\max},\ \max\left(0,\ S_{t-1} + I_t - D_t\right)\right).$$

Anything above capacity is lost as overflow spill, and any unmet demand is counted as a shortage day. Running this over 365 days of real rainfall shows how big a tank must be, which is much more convincing to a farmer than a single annual total.

2.1.4 Soil Fertility and Water Quality

Soil fertility depends on pH, available nitrogen, phosphorus and potassium (kg/ha), organic carbon and electrical conductivity (EC, a salinity measure). Organic carbon also governs how much water the soil holds, which links soil health to water conservation. Water for drinking is judged against the Bureau of Indian Standards norm IS:10500 [9] on pH, total dissolved solids, turbidity, nitrate and coliform bacteria. Water for irrigation is judged mainly on salinity and on the risk of clogging drip emitters, using criteria from the Central Pollution Control Board [10].

2.1.5 Machine Learning with Weka

Weka is an open-source Java library of machine-learning algorithms [2]. JalSathi uses two of them. Linear regression fits a straight-line trend to historical seasonal rainfall. Random Forest builds many decision trees on random subsets of the data and averages their output, which usually handles irregular patterns better. Both are trained on 115 years of India Meteorological Department (IMD) subdivision rainfall (1901 to 2015) [7] and used to estimate the coming season together with a historical variance band of about $\pm 18\%$.

2.1.6 Geographic Information Systems

A GIS places data on a map. India WRIS (Water Resources Information System) [8] publishes observation wells with their location, depth and water-table level. Showing these as colour-coded pins on a Leaflet.js map [4] (shallow below 5 m, moderate 5 to 12 m, deep above 12 m) lets a farmer see how stressed the local aquifer is.

2.1.7 Web Application Architecture

JalSathi follows a client–server design. The browser runs a vanilla JavaScript interface that calls a REST API over HTTP and exchanges JSON. REST uses plural resource paths and HTTP verbs: GET reads, POST creates and PUT updates. The server is layered into controllers (handle requests), services (business rules), repositories (database access) and entities (table mappings).

Spring Boot supplies dependency injection, meaning that the framework creates the objects a class needs and hands them over, rather than the class creating them itself [1]. Spring Data JPA maps Java classes to relational tables through Hibernate [6], so repository interfaces can save and find records without hand-written SQL. Spring Security with JSON Web Tokens (JWT) keeps the API stateless: each request carries a signed token that holds the user, role and expiry. Passwords are stored as BCrypt hashes, which are slow to guess and safe to store.

2.1.8 Relational Databases and MySQL

A relational database stores data in tables made of rows and columns, and links tables through keys. The data of this platform is naturally relational: a farmer owns tanks, a tank has many transactions, and each farmer has soil reports, water tests and alerts. MySQL [5] is used for production. For development the file-based H2 database is used, so that every team member can run the application without installing a database server.

2.1.9 Internationalisation and Inclusive Design

All interface text is held in two JSON files (Marathi and English) and loaded by a client-side script, so the language can be switched instantly without reloading the page. The assisted mode uses large buttons and short steps so that a Krushi Sahayak can fill in data on behalf of a farmer who is not comfortable with a smartphone.

2.2 Summary of the Investigation in the Published Sources

Because this is an application-development project, the study covered official datasets, standards and field research rather than only research papers. The key sources were:

- Field and agronomic studies by Pawar et al. (2014), Karkar and Bhalerao (2017), Patode et al. (2021), Garg et al. (2022) and Todmal (2019) [15, 16, 17, 18, 19], which report the yield, cost and income effects of farm ponds, check dams and rooftop harvesting in Maharashtra and the large share of runoff that is never harvested.
- IMD rainfall series and district normals [7], used for forecasting and for comparing a farmer's logged rain against the 30-year average.
- India WRIS observation-well data [8] and the CPCB and BIS IS:10500 water-quality norms [10, 9].
- Programme documents of the Paani Foundation and the NAAM Foundation, and the Maharashtra schemes *Magel Tyala Shettale* and *Jalyukt Shivar Abhiyan* [11, 12, 13].

- Official documentation for Spring Boot, Spring Security, Hibernate, Weka, Chart.js, Leaflet.js, OpenPDF and OpenCSV [1, 6, 2, 3, 4].

The study showed that the barriers are mostly knowledge and confidence, not a lack of rain or technology. This directly shaped the platform: every module turns a technical idea into a number or a verdict that the farmer can act on.

2.3 Comparison between the Tools, Methods and Algorithms

The team compared the candidate technologies before selecting the stack. The comparison criteria were learning curve, suitability for low-end phones and slow networks, ecosystem maturity and fit with a REST-based backend. Table 2.1 summarises the comparison.

Table 2.1: Comparison of candidate technologies

Layer	Option A	Option B	Selected and Reason
Backend	Node.js with Express	Django (Python)	Spring Boot (Java 17): strong typing, mature JPA and Security support, built-in test tooling
Database	MongoDB	MySQL only	MySQL 8 for production with file-based H2 for development, so the project runs without installing a server
Frontend	React or Angular	Plain static pages	Vanilla HTML, CSS and ES6: no build step, light on low-end phones and slow networks
Charts	D3.js	Google Charts	Chart.js 4.4: simple, responsive, works for bars, lines and doughnuts
Maps	Google Maps	OpenLayers	Leaflet.js with OpenStreetMap tiles: free, small, easy to use
Machine learning	Python scikit-learn service	External cloud ML API	Weka embedded in Java: one deployable application, no separate service
Authentication	Server sessions	Basic auth	Stateless JWT with BCrypt: suits a REST API

2.4 Algorithm(s) with Example

The core logic of the platform consists of five algorithms. Each is described with its pseudocode and the complexity analysis expected in the course.

2.4.1 Harvest Calculation

The harvest calculation converts the entered area to square metres, selects the runoff coefficient for the surface type and multiplies the three values.

```
ALGORITHM Harvest(area, unit, rainfall, surfaceType)
  areaSqm <- area * conversionFactor(unit)   // guntha 101.17, acre 4046.86
  C <- runoffCoefficient(surfaceType)        // 0.85, 0.60, 0.40, 0.30, 0.20
  litres <- areaSqm * rainfall * C
  RETURN litres, litres / 1000 (m3), litres / 1000 (tanks of 1,000 L)
END
```

Listing 2.1: Pseudocode: harvest calculation

Example: a 1,000 m^2 compacted yard ($C = 0.60$) with 700 mm of rain gives $1000 \times 700 \times 0.60 = 420{,}000$ litres. *Time complexity:* $O(1)$. *Space complexity:* $O(1)$.

2.4.2 365-Day Tank Simulation

The simulation walks through each day of a rainfall series, adds the inflow from the catchment, subtracts the daily demand and keeps the stock between zero and the tank capacity.

```
ALGORITHM SimulateTank(rainSeries[365], area, C, capacity, dailyDemand)
  stock <- 0 ; spill <- 0 ; shortageDays <- 0
  FOR day FROM 1 TO 365 DO
    inflow <- area * rainSeries[day] * C
    stock <- stock + inflow - dailyDemand
    IF stock > capacity THEN spill <- spill + (stock - capacity) ; stock <- capacity
    IF stock < 0 THEN shortageDays <- shortageDays + 1 ; stock <- 0
  END FOR
  RETURN stock, spill, shortageDays
END
```

Listing 2.2: Pseudocode: 365-day tank simulation

The capacity sweep repeats this run for $0.5\times$, $1.0\times$, $1.5\times$ and $2.5\times$ of the chosen capacity. *Time complexity:* $O(365 \cdot k)$ for k capacities, which is linear. *Space complexity:* $O(1)$ per run.

2.4.3 Soil Fertility Index

Each laboratory parameter is classified against an agronomic threshold, converted to weighted points and added to give a score between 0 and 100.

```
ALGORITHM FertilityIndex(ph, N, P, K, OC, EC)
  FOR each parameter p IN {ph, N, P, K, OC, EC} DO
    class(p) <- LOW | MEDIUM | HIGH | OPTIMAL     // by agronomic thresholds
    points(p) <- weight(p) * level(class(p))
  END FOR
  score <- SUM of points, scaled to 0..100
  band <- Poor | Moderate | Good | Excellent      // by score range
  tips <- advice for each parameter outside its optimal range
  RETURN score, band, tips
```

```
10 END
```

Listing 2.3: Pseudocode: soil fertility index

The thresholds used are: nitrogen low below 280, medium 280 to 560, high above 560 kg/ha; phosphorus low below 11, medium 11 to 25, high above 25 kg/ha; potassium low below 110, medium 110 to 280, high above 280 kg/ha; optimal pH 6.5 to 7.5. *Time and space complexity:* $O(1)$, since the number of parameters is fixed.

2.4.4 Water-Quality Verdict

A water sample is graded for drinking against the BIS IS:10500 limits, and separately for irrigation on salinity and emitter-clogging risk.

```
1 ALGORITHM WaterVerdict(ph, tds, turbidity, nitrate, coliform)
2   IF coliform = TRUE THEN drinking <- NOT_SAFE      // advise chlorination or boiling
3   ELSE
4     compare each value with its BIS IS:10500 limit
5     all within limit -> SAFE ; one near limit -> CAUTION ; any beyond limit -> NOT_SAFE
6   END IF
7   irrigation <- grade from salinity (TDS / EC) and emitter-clogging risk
8   RETURN drinking, irrigation, advice
9 END
```

Listing 2.4: Pseudocode: water-quality verdict

Time and space complexity: $O(1)$.

2.4.5 Payback Period

The economic module converts harvested water into rupees and divides the cost of the structure by the yearly saving:

$$\text{Tanker Savings} = \frac{\text{Harvested Litres}}{1000} \times \text{Cost per 1000 L},$$

$$\text{Payback (years)} = \frac{\text{Structure Capital Cost}}{\text{Tanker Savings} + \text{Pumping Savings}}.$$

Pumping savings equal the diesel or electricity cost per hour multiplied by the pumping hours saved. *Time and space complexity:* $O(1)$.

Chapter 3

Analysis and Design

3.1 Methodology / Procedure Adopted

The team followed an iterative and incremental development model. The project was split into small vertical slices (for example “login and roles”, “runoff calculator”, “tank ledger”, “365-day simulation”), and each slice was specified, built, tested and demonstrated before the next one started. This suited the project because the formulas, datasets and screens were refined after early demonstrations.

Weekly meetings. The team reviews the work done in the previous week, marks tasks in the Gantt chart as complete or delayed, and assigns the tasks for the coming week.

Version control. Work was committed in small steps with readable messages, so that each member’s changes could be reviewed.

Monitoring progress. Progress is measured with three indicators: the percentage of Gantt-chart tasks completed on time, the number of modules that pass their test checklist, and the number of open defects.

3.2 Analysis

3.2.1 Feasibility Study

The feasibility of the project was checked on three grounds.

- **Technical feasibility:** Java 17, Spring Boot, MySQL, H2, Weka, Chart.js and Leaflet.js are mature and open source, and the team had prior experience with Java, SQL and JavaScript.
- **Economic feasibility:** There is no licence cost. Development uses the file-based H2 database, and production needs only a free-tier MySQL server. The user needs only a smartphone; no sensor or equipment is required.
- **Operational feasibility:** The web application runs in any modern phone browser from 360 px width, is in Marathi by default, and has an assisted mode for field workers, so farmers need little training.

3.2.2 Modification of Requirements

Several requirements changed during the project, and the reasons are recorded here:

- **Live weather and IoT replaced by bundled datasets.** Real-time forecasts and LoRaWAN or GSM tank sensors need external services and hardware, so the prototype uses bundled IMD, WRIS and daily-weather datasets and manual entry. Both are recorded as future work.
- **Machine learning embedded in the backend.** Instead of a separate Python service, the Weka models run inside the Spring Boot application, which keeps deployment to a single program.
- **Dual database mode.** H2 was added for development beside MySQL so that every team member can run the project without installing a database server.
- **Assisted mode added.** After studying the user group, a big-button wizard for field workers was added to include farmers with low digital literacy.
- **Cost and subsidy module added.** It was added because the perceived high cost is one of the four barriers found in the survey.

3.2.3 Software / System Requirement Specification – IEEE Format

The complete software requirement specification written in IEEE format is attached as Appendix A. It defines the purpose, scope, user classes, functional requirements and non-functional requirements of the system.

3.3 System Architecture / Design

JalSathi uses a client–server, three-tier architecture. The client tier is a mobile-first web interface in the browser. The application tier is a Spring Boot REST service on port 8080 with a JWT filter, 20 controllers and an embedded Weka engine. The persistence tier is a MySQL 8 or H2 database with 23 tables, loaded at start-up with 8 reference datasets. Figure 3.1 shows the block diagram.

The main advantages of the proposed system over existing practice are:

- **numbers, not guesswork:** the calculator and the 365-day simulation size a structure from the farmer's own area, rainfall and tank capacity;
- **cost made visible:** tanker savings, payback and eligible schemes are shown in rupees and years;
- **advice in the farmer's language:** soil, water and crop advice is given in Marathi by default;
- **predictive:** a machine-learning forecast and alerts tell the farmer what is coming before the monsoon;

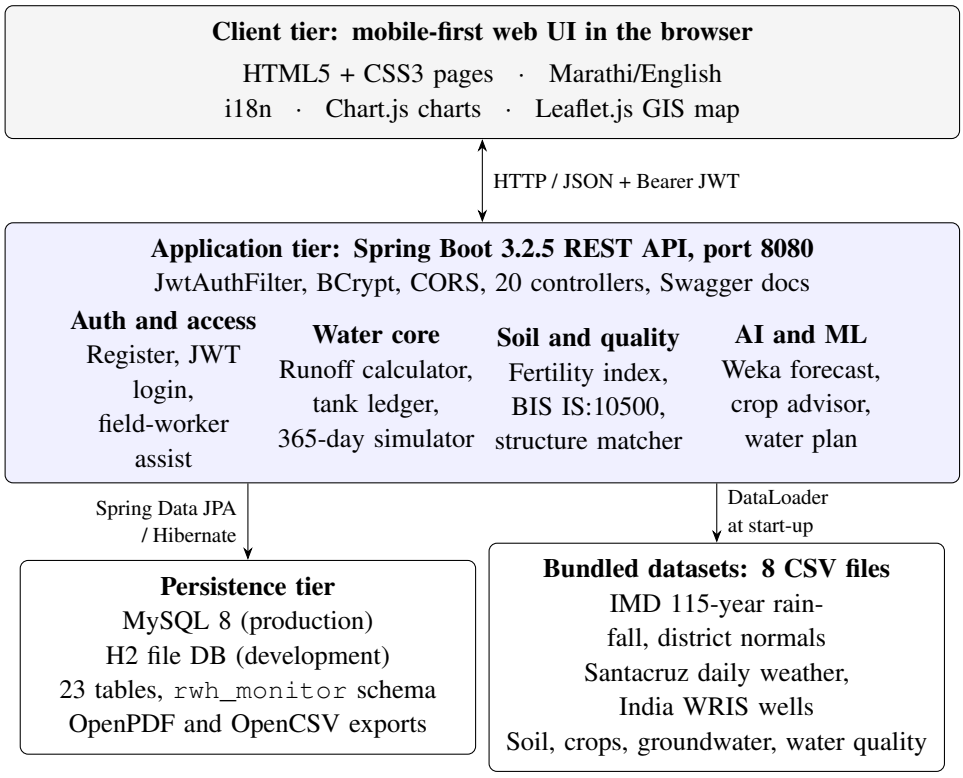


Figure 3.1: Three-tier architecture of JalSathi

- **inclusive:** an assisted mode lets a field worker act on behalf of a farmer with low digital literacy; and
- **extensible:** the layered backend and bundled datasets make it simple to add new modules, districts or live data feeds.

3.3.1 Modules and their Description

The system is divided into the modules listed in Table 3.1. Each module corresponds to a group of screens in the frontend and a set of controllers and services in the backend.

Table 3.1: Functional modules of JalSathi

Module	Description	Backend support
Authentication and RBAC	Phone-number login, JWT, roles Farmer, Field Worker and Admin	AuthController, SecurityConfig, JwtAuthFilter
Farmer dashboard	Tank gauge, rain versus 30-year normal, soil and maintenance tiles	Aggregates several services
Runoff calculator	Harvested litres from area, rainfall and surface type	HarvestController, HarvestService

Module	Description	Backend support
Rainfall logger	Daily rain entries compared with IMD district normals	RainfallController
Tank tracker	Tanks with inflow and outflow ledger and animated gauge	TankController, TankService
365-day simulation	Daily water balance and capacity sweep	SimulationController, SimulationService
Soil fertility	0 to 100 index, band and restoration advice	FertilityController, FertilityService
Structure recommender	Low-cost manual option first, mechanised option second	RecommendationController, RecommendationService
AI advisor	Weka monsoon forecast, crop picks, 5-month water budget	AiController, AiService
Groundwater map	WRIS wells as colour-coded pins on Leaflet	GroundwaterController, GroundwaterService
Water quality	BIS IS:10500 drinking and irrigation verdicts	WaterQualityController, WaterQualityService
Cost and subsidy	Tanker and pumping savings, payback, schemes	CostController, CostService
Knowledge base	Guides, community movements, research evidence	LearningController, ContentController
Assisted mode	Large-button registration wizard for field workers	AssistController, FieldWorkerService
Admin console	Taluka adoption, total harvest, structure mix, user list	AdminController, AdminService
Reports	PDF water-balance summary and CSV ledgers	ReportController, ReportService
Alerts	Low-tank, heavy-rain and desilting alerts	AlertController, AlertService

Use Cases

Three actors use the system. The **Farmer** registers, runs calculators and simulations, logs rain, tracks tanks, tests soil and water, reads advice and exports reports. The **Field Worker** registers farmers through the assisted wizard, surveys land and is assigned a set of farmers to support. The **Administrator** views adoption statistics, harvested totals, structure distribution and the user registry. Figure 3.2 shows the use case diagram.

Data Design

The relational model has 23 tables in four groups. The full entity-relationship diagram is shown in Figure 3.3 and the groups are summarised in Table 3.2.

[Insert screenshot here]
Use case diagram: actors Farmer, Field Worker and Admin with their modules

Figure 3.2: Use case diagram of JalSathi

[Insert screenshot here]
ER diagram of the 23 tables

Figure 3.3: Entity-relationship diagram of JalSathi

Table 3.2: Main database entities

Group	**Entities (tables)**	**Key attributes and relationships**
User management	users, farmer_profile	`users` holds phone (unique), name, password hash, role, language and location; `farmer_profile` is one-to-one with `users` and links an assigned field worker
Farmer assets and hydrology	land_data, tank, tank_transaction, harvest_calculation, rainfall_record, tank_simulation	Each belongs to a farmer; `tank_transaction` belongs to a tank and records type (inflow or outflow), purpose, litres and date
Advisory and analytics	fertility_report, recommendation, ai_prediction, alert, maintenance_task, water_quality_test	Each belongs to a farmer; `maintenance_task` and `water_quality_test` can also link to a tank; `ai_prediction` stores input, output, confidence and model version
Master reference data	crop_profile, district_rainfall_normal, historical_rainfall, daily_weather, groundwater_reading, district_region_map, learning_guide, partner_organization, research_evidence	Standalone tables loaded from CSV files and seeders; they carry Marathi and English text where needed

REST API Design

The backend exposes its REST endpoints through 17 feature controllers (20 in all with the supporting ones). Resource names are plural nouns, JSON is used for data, and every protected endpoint needs a Bearer JWT. The full, always up-to-date list can be browsed in Swagger UI at `/swagger-ui.html`. Table 3.3 groups the controllers.

Table 3.3: REST controllers exposed by the backend

Controller	Purpose
AuthController	Register and log in; returns a signed JWT
HarvestController	Compute and store harvest calculations
RainfallController	Log daily rain and compare with district normals
TankController	Create tanks, post inflow and outflow, read level
SimulationController	Run and save the 365-day simulation
FertilityController	Compute and store soil fertility reports
RecommendationController	Suggest RWH structures and capacities
AiController	Monsoon forecast, crop recommendation, water plan
GroundwaterController	Serve WRIS well readings for the map
WaterQualityController	Record tests and return verdicts
CostController	Savings and payback calculation
LearningController, ContentController	Guides, community and research content
AssistController	Assisted farmer registration
AdminController	Statistics and user registry for admins
ReportController	PDF and CSV export
AlertController	List and mark alerts

3.4 Detailed Design Rationale

This section explains why each part of the design was chosen and how the parts cooperate. The choices follow directly from the requirements in Appendix A and the theory in Chapter 2.

3.4.1 Frontend Design

The frontend is a set of plain HTML pages (for example `dashboard.html`, `calculator.html`, `tank.html`, `simulation.html`, `fertility.html`, `ai-assist.html`, `groundwater.html`, `water-quality.html`, `cost.html`, `assist.html`, `admin.html` and `reports.html`) that share one stylesheet built on CSS variables and a glassmorphism card style, plus small JavaScript modules that call the API with `fetch`. The language files `frontend/i18n/mr.json` and

`frontend/i18n/en.json` hold all text. The layout works from 360 px width, so it is usable on low-end phones.

3.4.2 Authentication and Role-Based Access

The 10-digit mobile number is the username. After a successful login the server returns a JWT with the user, role and expiry. `JwtAuthFilter` checks the token on every request and `SecurityConfig` restricts routes by role: administrator endpoints are open only to `ADMIN`, and assisted-registration endpoints only to `FIELD_WORKER`. Passwords are stored as BCrypt hashes (cost factor 10), and sessions are stateless.

3.4.3 Backend Layered Design

The backend separates its responsibilities into layers, each with one job.

- **Controllers** (package `com.dbit.rwh.controller`) receive requests, validate input and call services.
- **Services** hold the business rules: formulas, simulation, scoring, verdicts and forecasting.
- **Repositories** (Spring Data JPA) give access to the database tables and contain no business logic.
- **Entities** map Java classes to the 23 tables.
- **DataLoader** loads the eight CSV datasets and the demonstration accounts on first start.
- **Config and security classes** set up CORS, JWT filtering and the OpenAPI documentation.

3.4.4 Sequence of a Harvest Calculation

1. The farmer enters area (with unit), rainfall and surface type on the calculator page.
2. The browser sends the data as JSON with the Bearer token.
3. `HarvestController` passes it to `HarvestService`, which converts units, picks the runoff coefficient and multiplies the three values.
4. The result is saved as a `harvest_calculation` row linked to the farmer.
5. The response returns litres, cubic metres and the number of 1,000-litre tanks filled.
6. The page shows the result and adds it to the saved history.

3.4.5 Sequence of a Monsoon Forecast

`AiService` loads the historical rainfall for the farmer's subdivision (through the district-to-region map), trains or reuses the Weka linear-regression and Random Forest models,

predicts the coming season and adds the historical variance band. The result is stored in `ai_prediction` with its confidence and model version, and returned to the browser for display.

3.4.6 Non-Functional Design Decisions

Responsiveness comes from a mobile-first layout. Usability comes from Marathi as the default language and from the assisted mode. Security comes from JWT, BCrypt and role checks. Portability comes from Java and the dual H2 and MySQL set-up. Maintainability comes from the layered structure and the naming conventions of Chapter 4.

Chapter 4

Implementation

4.1 Implementation Plan

The implementation was planned over twelve weeks. Figure 4.1 shows the Gantt chart, and the Work Breakdown Structure with the individual contribution of each team member is given in Table 4.1.

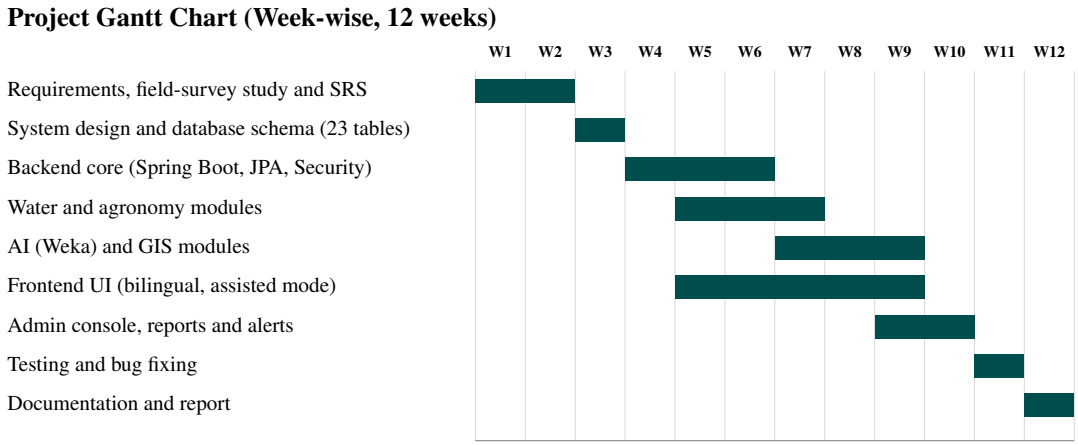


Figure 4.1: Gantt chart of the JalSathi project (12 weeks)

Table 4.1: Work Breakdown Structure and individual contribution

WBS	Work Package	Owner	Contribution
1	Requirements, SRS and system design	All members	–
1.1	Field-survey study, requirement gathering and SRS (Appendix A)		
1.2	Database schema and ER design (23 tables)		
2	Backend core (Spring Boot, JPA, Security)		
2.1	Entities, repositories, DataLoader and CSV datasets		

WBS	Work Package	Owner	Contribution
2.2	JWT authentication, role-based access, BCrypt		
3	Water and agronomy modules		
3.1	Harvest calculator, rainfall logger and tank ledger		
3.2	365-day simulation, fertility index, water quality		
4	AI, GIS and economics		
4.1	Weka monsoon forecast, crop and water-budget advisor		
4.2	Leaflet groundwater map, cost and subsidy calculator		
5	Frontend, reports, testing and documentation		
5.1	Bilingual UI, dashboard, assisted mode, admin console		
5.2	PDF and CSV reports, alerts, testing, project report		

4.2 Coding Standard

The team agreed on the following conventions before coding started, so that the code of all members looks like a single project.

4.2.1 Naming Conventions

Table 4.2 lists the naming conventions used in the project.

4.2.2 Comments, Structure and Complexity

Public classes and non-trivial methods carry a short comment explaining their purpose. Business rules such as runoff coefficients, fertility thresholds and BIS limits are kept in named constants in the service classes rather than scattered through the code. Controllers contain no business logic beyond validation and calling a service, so that the layers remain separate. Entities use annotations for mapping, and request and response data use separate classes so that the database structure is not exposed directly to the browser. For every algorithm the expected time and space complexity is written beside its pseudocode in Chapter 2.

Listing 4.1 shows an excerpt of the harvest calculation, which illustrates the naming and the use of named constants.

Table 4.2: Naming conventions used in the project

Element	Convention	Example
Java package	lowercase, grouped by layer under `com.dbit.rwh`	controller, service, entity, config, security
Java class	PascalCase with a layer suffix	HarvestController, HarvestService, TankTransaction
Java method and variable	camelCase; verb first for methods	calculateHarvest(), runoffCoefficient
Java constant and enum	UPPER_SNAKE_CASE	FARMER, FIELD_WORKER, ADMIN
REST endpoint	lowercase, plural nouns, JSON	/api/... resources
Database table and column	lowercase with underscores	tank_transaction, land_area_sqm
Frontend file	lowercase with hyphens	water-quality.html, ai-assist.html
JavaScript function and variable	camelCase	loadTankLevel, authToken
CSS	custom properties for colours and spacing	–primary, –card-bg
Language keys	dotted keys in mr.json and en.json	dashboard.title

[Paste a 10 to 15 line code excerpt here]
`HarvestService.java`: the method that converts the area unit, picks the runoff coefficient and returns the litres

Listing 4.1: Excerpt from HarvestService.java: computing harvested litres

4.3 Testing

4.3.1 Testing Methods Used

The following methods were used to verify the project.

- **Automated backend tests:** run with the Maven wrapper (`mvnw test`). The suite starts the Spring context and checks services and endpoints.
- **API testing:** every endpoint was called from Swagger UI at `/swagger-ui.html` with and without a token, and the JSON responses were checked against the expected structure.
- **Functional (black-box) testing:** each page was tested in the browser against the SRS, using the seeded demonstration accounts.
- **Negative testing:** empty fields, negative areas, impossible pH values and calls without a token or with the wrong role were tried deliberately.
- **Calculation verification:** formula outputs were checked by hand against worked examples, for instance $1000 \times 700 \times 0.60 = 420{,}000$ litres.
- **Responsive testing:** pages were checked at 360 px width on a phone and on a desktop browser.

4.3.2 Test Cases for Modules and Components

Table 4.3 lists the principal test cases. The expected results are taken directly from the application's behaviour as implemented, and the Status column is to be filled in after each run.

Table 4.3: Test cases for modules and components

ID	Module / Input	Expected Result	Type	Status
TC-01	Log in with a valid farmer phone number and password	Dashboard opens with a JWT stored in the browser	Functional	
TC-02	Log in with a wrong password	Error message; no token issued	Negative	
TC-03	Open a protected API without a token	Request rejected (unauthorised)	Security	
TC-04	Farmer opens an admin page or endpoint	Access denied	Security	
TC-05	Runoff calculator: 1,000 m^2, 700 mm, compacted yard	420,000 litres, 420 m^3 shown and saved to history	Functional	
TC-06	Calculator with 10 guntha	Area converted to 1,011.7 m^2 before calculation	Functional	

ID	Module / Input	Expected Result	Type	Status
TC-07	Tank ledger: post an inflow then an outflow	Level and percentage update; gauge animates	Functional	
TC-08	Tank outflow larger than stored water	Rejected or flagged; level never goes below zero	Negative	
TC-09	365-day simulation with four capacity multipliers	Harvest, spill and shortage days returned for each size	Functional	
TC-10	Soil fertility with pH 6.8 and medium NPK	Score 0 to 100, a band and Marathi/English tips	Functional	
TC-11	Water test with coliform present	Drinking verdict Not Safe with chlorination or boiling advice	Functional	
TC-12	AI monsoon forecast for a district	Forecast with $\pm$18% variance band; saved in `ai_prediction`	Integration	
TC-13	Groundwater page	Wells shown as shallow, moderate and deep pins	Functional	
TC-14	Cost calculator with an Rs. 8,000 structure	Annual savings and payback in years	Functional	
TC-15	Field worker registers a farmer in assisted mode	Farmer account created and linked to that field worker	Functional	
TC-16	Download PDF and CSV reports	Files open with correct totals	Functional	
TC-17	Switch language to Marathi and back	All labels change without reload	Functional	
TC-18	Run `mvnw test`	All tests pass	Build	

4.3.3 Results of Testing and System Performance

Automated tests. The backend test suite ran 14 tests with 0 failures and 0 errors, so 100% of the tests passed.

Endpoint testing. Figure 4.2 shows a response captured from Swagger UI.

Integration results. The browser pages and the REST API are connected through the `fetch` API, and the frontend reads every value shown on screen from the backend. The screens and their behaviour under the test cases of Table 4.3 are shown in Chapter 5.

[Insert screenshot here]
Swagger UI showing one endpoint response (e.g. harvest calculation or tank endpoint) with the JSON result

Figure 4.2: REST response captured in Swagger UI

Chapter 5

Results and Discussion

This chapter presents the output of the running application. Each figure is a screenshot taken while the application was running with the seeded demonstration accounts, and the discussion explains what each result shows against the requirements and the test cases of Chapter 4.

5.1 Intermediate Results

The intermediate results were the individual vertical slices completed during the project: authentication and roles, the runoff calculator, the tank ledger, the 365-day simulation, the soil and water-quality modules, and the AI and GIS modules. Each slice was demonstrated to the team and tested before the next slice was started.

5.2 Final Results and Analysis

5.2.1 Landing Page, Login and Language Toggle

The landing page (Figure 5.1) introduces the platform and offers login and registration. Users log in with a 10-digit mobile number and a password, and the page opens in Marathi by default with an instant English toggle (Figure 5.2), as required by test cases TC-01, TC-02 and TC-17. A wrong password gives an error and no token is issued.

5.2.2 Farmer Dashboard

After login the farmer sees the central dashboard (Figure 5.3): the live tank gauge, a tile comparing the farmer's logged rain with the 30-year IMD district normal, a soil-health summary and upcoming maintenance reminders, with shortcuts to the calculators. It meets the need for one place that shows the farmer's water position at a glance.

5.2.3 Runoff and Harvesting Calculator

The calculator (Figure 5.4) takes the catchment area (in square metres, gunthas or acres), the rainfall in mm and the surface type, applies the runoff coefficient and returns harvested litres, cubic metres and the number of 1,000-litre tanks filled. For 1,000 m^2 of compacted yard and

[Insert screenshot here]
Landing / login page in Marathi, logged out

Figure 5.1: Landing and login page (Marathi default)

[Insert screenshot here]
The same page after clicking the language toggle

Figure 5.2: Language toggle showing the English interface

[Insert screenshot here]
Dashboard for the Pune farmer 7777777701

Figure 5.3: Farmer dashboard

700 mm of rain it returns 420,000 litres, which matches the hand calculation (TC-05, TC-06). Every run is saved in the history.

[Insert screenshot here]
Calculator with the 1,000 m^2, 700 mm,
compacted-yard example and the result card

Figure 5.4: Runoff and harvesting calculator

5.2.4 Rainfall Logger and District Normals

The farmer logs daily rain, and a bar chart (Figure 5.5) compares the totals with the 30-year IMD normal for the district. The page shows whether the season is running in deficit, normal or surplus, which helps the farmer decide when to irrigate and when to conserve.



Figure 5.5: Daily rainfall log against district normals

5.2.5 Tank Storage Tracker

The tank page (Figure 5.6) keeps a double-entry ledger of inflows (rain harvest, transfer) and outflows (irrigation, drinking), and shows an animated gauge with the current volume and percentage full. An alert is raised when the level drops below 20% (TC-07, TC-08).

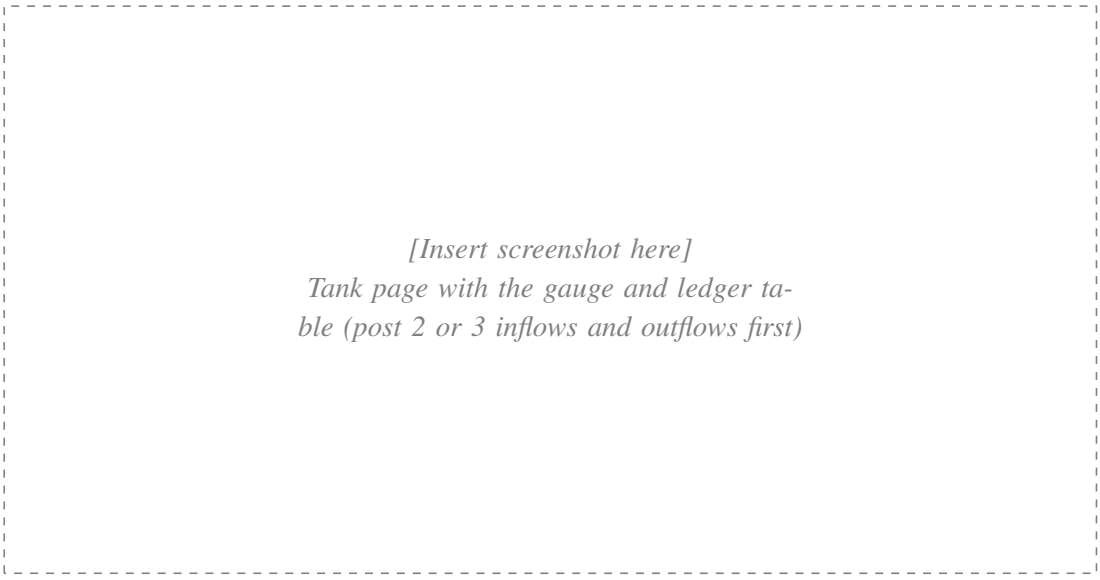


Figure 5.6: Tank tracker with gauge and transaction ledger

5.2.6 365-Day Tank Simulation

The simulation (Figure 5.7) runs the daily water balance for a full year on real daily rainfall and reports total harvest, monsoon and dry-season collection, overflow spill and the exact number of shortage days. The capacity sweep repeats the run at 0.5×, 1.0×, 1.5× and 2.5× capacity, so the farmer can see how a bigger tank removes shortage days (TC-09).

[Insert screenshot here]
Simulation result with the capacity comparison table visible

Figure 5.7: 365-day simulation and capacity sweep

5.2.7 Soil Fertility Index

The farmer enters laboratory values for pH, N, P, K, organic carbon and EC. The system (Figure 5.8) returns a 0 to 100 score, a band (Poor, Moderate, Good or Excellent) and bilingual advice such as gypsum for alkaline soil, lime for acidic soil, farmyard manure and bio-fertilisers (TC-10).

[Insert screenshot here]
Fertility form and the score and advice card (use pH 6.8 with medium NPK)

Figure 5.8: Soil fertility index and advice

5.2.8 Structure Feasibility and Recommendation

The recommender (Figure 5.9) uses land area, irrigation source, existing structure and soil to suggest a structure. It always shows the low-cost manual option first (a lined Jalkund, a recharge pit, shramdaan trenches costing about Rs. 3,000 to Rs. 8,000) and the mechanised,

subsidy-eligible option second.

[Insert screenshot here]
Structure recommendation page (login as farmer 7777777703, Latur, no structure)

Figure 5.9: Structure feasibility and dual-track recommendation

5.2.9 AI Climate and Crop Water-Budget Advisor

The advisor (Figure 5.10) predicts the coming season's monthly rainfall with the Weka models and shows the $\pm 18\%$ historical variance band. It then ranks suitable Kharif, Rabi and Zaid crops by soil, district and water need, and prints a five-month storage-versus-demand plan with deficit warnings and mulching advice (TC-12).

[Insert screenshot here]
AI Assist page showing forecast, crops and water plan

Figure 5.10: AI climate and crop water-budget advisor

5.2.10 GIS Groundwater Monitoring

The map (Figure 5.11) shows India WRIS observation wells for Maharashtra as colour-coded pins: shallow (below 5 m), moderate (5 to 12 m) and critical (above 12 m). Selecting a pin shows the well depth, water-table level and aquifer type (TC-13).

[Insert screenshot here]
Groundwater map zoomed to Maharashtra with one pin popup open

Figure 5.11: Groundwater observation wells on the Leaflet map

5.2.11 Water Quality Assessment

The water-quality page (Figure 5.12) grades a sample on pH, TDS or EC, turbidity, nitrate and coliform, giving separate drinking and irrigation verdicts of Safe, Caution or Not Safe. A sample with coliform bacteria is graded Not Safe for drinking, with advice to chlorinate or boil for 10 minutes (TC-11).

[Insert screenshot here]
Water-quality form with the verdict card (use a coliform-present sample so the Not Safe advice is visible)

Figure 5.12: Water-quality test and BIS IS:10500 verdict

5.2.12 Economic Payback and Subsidies

The cost calculator (Figure 5.13) converts harvested litres into tanker savings, adds pumping savings and divides the structure cost by the total annual savings to give the payback period. It also lists the Magel Tyala Shettale and Jalyukt Shivar schemes, which answers the "high perceived cost" barrier directly (TC-14).

[Insert screenshot here]
Cost page with payback result and subsidy information (use an Rs. 8,000 structure)

Figure 5.13: Cost, payback and subsidy calculator

5.2.13 Knowledge Base, Community Movements and Research Evidence

Bilingual illustrated guides explain farm-pond lining, first-flush separators, mulching and pre-monsoon desilting (Figure 5.14). The community page presents the Paani Foundation and the NAAM Foundation, and the evidence page summarises the peer-reviewed studies (Figure 5.15).

[Insert screenshot here]
A learning guide page (optionally the Marathi version)

Figure 5.14: Illustrated guides in the knowledge base

[Insert screenshot here]
Community or Evidence page (either one)

Figure 5.15: Community movements and research evidence

5.2.14 Assisted Mode for Field Workers

The assisted wizard (Figure 5.16) uses large buttons and short steps so that a Krushi Sahayak can register a farmer, record land and crop details, and assign monitoring alerts on the farmer's behalf (TC-15).

[Insert screenshot here]
Assisted wizard step, logged in as field worker 8888888888

Figure 5.16: Big-button assisted registration wizard

5.2.15 Administrator Console

The admin console (Figure 5.17) shows taluka-level adoption, the total litres harvested, the distribution of structure types and a paginated registry of farmers. It lets the administrator see where adoption is low and which structures are most used.

[Insert screenshot here]
Admin dashboard, logged in as admin 9999999999

Figure 5.17: Administrator dashboard

5.2.16 Reports and Alerts

The report page exports a vector PDF containing the water balance, rainfall totals, tank ledger and soil advice, and CSV files of daily rainfall and transactions (TC-16), shown in Figure 5.18. The alert page (Figure 5.19) lists low-tank, heavy-rain (above 65 mm) and desilting reminders.

[Insert screenshot here]
The generated PDF report opened in a PDF viewer (page 1)

Figure 5.18: Exported PDF water-budget report

5.2.17 Comparison: Existing Practice and Proposed System

Table 5.1 compares the proposed system with the existing practices described in Chapter 1.

[Insert screenshot here]
Alerts page (lower a tank below 20% first so an alert appears)

Figure 5.19: Smart alerts and notifications

Table 5.1: Comparison of existing practice and JalSathi

Parameter	**Existing Practice**	**JalSathi**
Sizing a structure	Guesswork or a visit by an expert	Formula-based calculator and 365-day simulation on the farmer's own data
Cost and payback	Not known; assumed to be very high	Savings and payback in years, with subsidy information
Rainfall outlook	Generic forecasts	Weka forecast on 115 years of IMD data with a variance band
Soil and water testing	Lab report that the farmer cannot interpret	Instant score, verdict and advice in Marathi
Groundwater information	Not available to the farmer	Interactive WRIS well map
Maintenance	Forgotten	Desilting and low-tank alerts
Reach	Needs literacy and a smartphone	Marathi default and an assisted mode for field workers
Monitoring by NGOs and officials	Manual surveys	Admin dashboard and PDF/CSV exports

5.2.18 Interpretation of the Results

The screenshots allow the behaviour of the platform to be read against the theory in Chapter 2. The calculator and the simulation turn the runoff formula and the water-balance equation into numbers the farmer can use: litres harvested, days of shortage and the benefit of a larger tank. The fertility index and the water-quality verdict turn laboratory values into a plain rating with a next step. The forecast, the groundwater map and the alerts tell the farmer what is coming and what to do before the monsoon. The values on the screens come from seeded demonstration farmers across five agro-climatic zones (Pune, Nagpur, Latur, Kolhapur and Jalna), so they illustrate how the system works; they are not measurements from real farms.

5.2.19 Discussion and Limitations

The results show that the platform meets its main objectives as a working prototype. However, the analysis also identified limitations that a responsible report must state:

- **Rainfall data source.** The 365-day simulation uses the Mumbai Santacruz daily rainfall record, which may differ from rain at an inland farm. A district-wise daily series would make it more accurate.
- **Manual data entry.** Rainfall and tank levels are typed in by the farmer or field worker; there are no sensors yet.
- **Forecast granularity.** The Weka models work at the level of an IMD subdivision, not a single village, and a forecast is only an estimate with a wide band.
- **Bundled datasets.** The data is loaded at start-up and is not refreshed automatically from IMD or WRIS.
- **No field trial yet.** The system has been tested with demonstration data and has not yet been evaluated with a large group of real farmers.
- **Basic alerts.** Alerts appear on the dashboard; SMS and WhatsApp delivery are future work.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

JalSathi shows that a smartphone can replace costly field equipment for rainwater-harvesting planning. The project delivers a bilingual, mobile-first platform with 20 functional modules: a runoff calculator, a tank ledger, a 365-day water-balance simulation, a soil fertility index, a BIS IS:10500 water-quality checker, a Weka-based monsoon forecast and crop advisor, a GIS groundwater map, a cost and subsidy calculator, an assisted mode for field workers, an admin console and PDF/CSV reports. It rests on a layered Spring Boot backend with stateless JWT security, 23 database tables and 8 bundled datasets, and its 14 automated tests all pass.

The project met its aim at the level of a working prototype. Each of the four barriers found in the farmer survey is answered by a specific feature: forecasting and alerts for rainfall uncertainty, calculators and recommenders for missing knowledge, the payback and subsidy module for perceived cost, and maintenance reminders for neglect. The platform supports UN SDG 6, 13 and 15. Its main gaps are the lack of live weather and sensor data and the absence of a field trial with real farmers, and these lead the future work.

6.2 Learning from Project Management

The team learned several lessons that will be useful in later projects.

- **Agree on data and formulas early.** Hydrological formulas, thresholds and units (guntha, acre, mm) had to be fixed with the supervisor before coding, so that all modules gave the same answers.
- **Control scope with a finalised specification.** Features such as IoT sensors and live weather were attractive but outside the semester. Writing the SRS first helped the team defer them.
- **Keep the user in mind.** Designing for low-literacy users led to the Marathi default and the assisted mode, which changed the interface more than any technical choice.
- **Separate the layers.** Keeping controllers, services and repositories apart made it easy for several members to work in parallel and to fix bugs quickly.
- **Commit small and often.** One commit per task, with a clear message, made it easy to

find when a bug appeared and to review each other's work.

- **Verify with evidence.** A hand-checked calculation, an API response or a test report is needed before a feature is called finished.
- **Protect secrets.** Signing keys and database passwords belong in environment variables, not in source code.

6.3 Future Work

The following enhancements are planned:

- **IoT sensor integration:** LoRaWAN or GSM ultrasonic tank-level sensors and soil-moisture probes for automatic telemetry.
- **Live weather hooks:** short-term forecasts from Open-Meteo or the IMD Mausam API.
- **IVR and WhatsApp bot:** a Marathi voice and chat bot for farmers on 2G feature phones.
- **Subsidy integration:** direct linking to the MahaDBT portal for Magel Tyala Shettale applications.
- **District-wise daily rainfall** for the simulation, instead of a single city record.
- **Field trial** with farmers and field workers in at least one taluka, to measure real adoption and water saved.
- **SMS alerts and cloud deployment** of the backend on a managed MySQL service.

Chapter A

Software Requirement Specification (IEEE Format)

A.1 Introduction

A.1.1 Purpose

This document specifies the requirements of JalSathi, a bilingual web platform for rainwater-harvesting planning and water-conservation monitoring. It is written for the development team, the supervisor and the evaluators.

A.1.2 Scope

The system provides harvest and tank calculators, a 365-day simulation, soil and water-quality assessment, machine-learning forecasts, a groundwater map, cost and subsidy estimates, an assisted registration mode, an administrator console and report export. It does not read live sensors or live weather services.

A.1.3 Definitions

RWH	Rainwater Harvesting
IMD	India Meteorological Department
WRIS	Water Resources Information System (India WRIS)
BIS IS:10500	Bureau of Indian Standards norm for drinking water
JWT	JSON Web Token used for stateless login
RBAC	Role-Based Access Control
EC	Electrical Conductivity (salinity), in dS/m
Krushi Sahayak	Agricultural field worker who assists farmers
Guntha	Land unit; 1 guntha = 101.17 m^2
REST	Representational State Transfer, the style of the backend API

A.2 Overall Description

A.2.1 Product Perspective

JalSathi is a new, self-contained product. A browser interface written in HTML, CSS and JavaScript calls a Spring Boot REST service by JSON over HTTP, and the service stores data in MySQL or H2.

A.2.2 User Classes and Characteristics

- **Farmer:** a small or marginal farmer with a smartphone, often with low digital literacy, who prefers Marathi.
- **Field worker (Krushi Sahayak):** assists farmers and registers them through the assisted wizard.
- **Administrator:** monitors adoption statistics and manages users.

A.2.3 Operating Environment

A modern browser (Chrome, Edge, Firefox or Safari) on a phone or desktop. Server side: Java 17, Maven, and MySQL 8 or the embedded H2 database.

A.2.4 Assumptions and Dependencies

The user has internet access, and rainfall and tank readings are entered manually. The map tiles come from OpenStreetMap and the charts from a CDN, so they need an internet connection.

A.3 Specific Requirements

A.3.1 Functional Requirements

ID	Requirement
FR-1	The system shall register and authenticate users by a 10-digit phone number and password and issue a JWT.
FR-2	The system shall support the roles Farmer, Field Worker and Admin and restrict endpoints by role.
FR-3	The system shall compute harvested litres from area, rainfall and runoff coefficient, with guntha and acre conversion, and save the history.
FR-4	The system shall let a farmer log daily rainfall and compare it with the 30-year district normal.

ID	Requirement
FR-5	The system shall let a farmer create tanks and record inflow and outflow transactions and show the current level.
FR-6	The system shall run a 365-day tank simulation and a capacity sweep and report harvest, spill and shortage days.
FR-7	The system shall compute a 0 to 100 soil fertility score and band with advice.
FR-8	The system shall grade a water sample for drinking and irrigation as Safe, Caution or Not Safe.
FR-9	The system shall recommend RWH structures, crops and a seasonal water budget.
FR-10	The system shall forecast seasonal rainfall with machine-learning models and show a variance band.
FR-11	The system shall show groundwater observation wells on a map.
FR-12	The system shall compute tanker savings, pumping savings and payback period.
FR-13	The system shall let a field worker register farmers through an assisted wizard.
FR-14	The system shall generate alerts for low tank level, heavy rain and maintenance due.
FR-15	The system shall show administrator statistics and a paginated user list.
FR-16	The system shall export reports as PDF and CSV.
FR-17	The system shall let the user switch the interface between Marathi and English.

A.3.2 Non-Functional Requirements

- **Usability:** a new farmer should complete the first calculation with the help of the Marathi interface alone.
- **Performance:** calculators and tank updates shall respond without perceptible delay on a normal mobile connection.
- **Responsiveness:** the layout shall work from 360 px width upward.
- **Security:** passwords shall be stored as BCrypt hashes, sessions shall be stateless, and protected endpoints shall reject requests without a valid token.
- **Maintainability:** the code shall follow the naming conventions of Chapter 4 and the controller, service, repository and entity layering.
- **Portability:** the application shall run on any operating system that supports Java 17.

Chapter B

Installation Procedure: Development Software

B.1 Software Required

- Java Development Kit 17
- Maven (the project includes the Maven wrapper `mvnw`)
- MySQL Server 8 with a schema named `rwh_monitor` (optional; H2 works without it)
- A code editor such as IntelliJ IDEA or Visual Studio Code
- A modern web browser

B.2 Running the Application

Method A, one-click menu. Run `run.bat` in the project root and choose 1 to start the backend (port 8080), 2 to start the frontend (port 3000) or 3 to start both.

Method B, IntelliJ IDEA.

1. Open the `backend` folder as a project; Maven imports the dependencies from `pom.xml`.
2. Run `RwhMonitorApplication.java`.
3. Open `http://localhost:8080` for the web interface.
4. Open `http://localhost:8080/swagger-ui.html` for the API documentation.
5. Open `http://localhost:8080/h2-console` for the development database (JDBC URL `jdbc:h2:file:./data/rwh_db`).

Method C, command line.

1. In the `backend` folder run `./mvnw spring-boot:run` (or `mvnw.cmd spring-boot:run` on Windows).

2. In a second terminal, in the `frontend` folder, run `run.bat`.

To run the automated tests, open a terminal in the `backend` folder and run `mvnw test`.

B.3 Demonstration Accounts

On first start `DataLoader` seeds the accounts in Table B.1 for testing. They are demonstration accounts only and must be changed or removed in any real deployment.

Table B.1: Seeded demonstration accounts

Role	**Phone (username)**	**Password**	**Profile**
Admin	9999999999	admin123	DBIT administrator
Field worker	8888888888	field123	Suresh Patil, Baramati (Pune)
Farmer	7777777701	farmer123	Dnyaneshwar Shinde, Shirur (Pune); rain-fed, farm pond, soybean
Farmer	7777777702	farmer123	Ananda Gaikwad, Ramtek (Nagpur); borewell, tank, cotton
Farmer	7777777703	farmer123	Balasaheb Jadhav, Ausa (Latur); rain-fed, no structure, tur
Farmer	7777777704	farmer123	Sunita Deshmukh, Shirol (Kolhapur); well, farm pond, sugarcane
Farmer	7777777705	farmer123	Vilasrao Pawar, Ambad (Jalna); rain-fed, no structure, jowar

Chapter C

Data Sheets: Electronic Components

Not applicable. JalSathi is a purely software project and does not use any electronic component.

References

Bibliography

[1] Broadcom Inc., “Spring Boot Reference Documentation,” Spring Framework. [Online]. Available: https://docs.spring.io/spring-boot/

[2] University of Waikato, “Weka: Machine Learning Software in Java.” [Online]. Available: https://ml.cms.waikato.ac.nz/weka/

[3] Chart.js Contributors, “Chart.js Documentation.” [Online]. Available: https://www.chartjs.org/docs/

[4] V. Agafonkin, “Leaflet: an open-source JavaScript library for interactive maps.” [Online]. Available: https://leafletjs.com

[5] Oracle Corporation, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] Red Hat, “Hibernate ORM Documentation.” [Online]. Available: https://hibernate.org/orm/documentation/

[7] India Meteorological Department (IMD), Ministry of Earth Sciences, Government of India, rainfall data and district normals. [Online]. Available: https://mausam.imd.gov.in

[8] Ministry of Jal Shakti, Government of India, “India WRIS: Water Resources Information System.” [Online]. Available: https://indiawris.gov.in

[9] Bureau of Indian Standards, “IS 10500: Drinking Water Specification.”

[10] Central Pollution Control Board (CPCB), water-quality criteria and norms. [Online]. Available: https://cpcb.nic.in

[11] Paani Foundation, “Satyamev Jayate Water Cup.” [Online]. Available: https://www.paanifoundation.in

[12] NAAM Foundation, Maharashtra. [Online]. Available: https://naamfoundation.org

[13] Government of Maharashtra, “Magel Tyala Shettale” and “Jalyukt Shivar Abhiyan” scheme documents.

[14] United Nations, “Sustainable Development Goals 6, 13 and 15.” [Online]. Available: https://sdgs.un.org

[15] Pawar et al., 2014, study on rainwater harvesting in Maharashtra. [Full citation to be completed.]

[16] Karkar and Bhalerao, 2017, farm-pond cost and payback. [Full citation to be completed.]

[17] Patode et al., 2021. [Full citation to be completed.]

[18] Garg et al., 2022. [Full citation to be completed.]

[19] Todmal, 2019, runoff and harvesting in Maharashtra. [Full citation to be completed.]

Acknowledgement

We express our sincere gratitude to our project supervisor, Asst. Prof. Shiv Negi, for his continuous guidance, patience and constructive feedback throughout the project. We thank the Head of the Department of Information Technology, Prof. Sunantha G., and the Principal, Dr. Sudhakar Mande, for providing the facilities and the encouraging environment in which this work was carried out. We also thank our faculty members and laboratory staff for their support, and our families and friends for their motivation.